



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

**Análise de Qualidade
Padrões e Diretrizes
Equipe 03 - Jogo Desconecta**

Ana Lúcia de Souza - 5084
Gustavo Eufrausino de Medeiros - 5062

Florestal
2025

Sumário

1.Introdução.....	3
2.Ata de Reunião.....	3
3.Padrões de codificação em Java.....	4
3.1.Comentários.....	4
3.2.Indentação e Espaçamento.....	6
3.3.Convenções de nomenclatura.....	6
3.4.Composição de Classes.....	9
3.5.Regras de formatação.....	9
4.Padrões para Banco de Dados.....	10
4.1.Nomenclatura do banco de dados:.....	10
4.2.Nomenclatura de Tabelas:.....	11
4.3.Nomenclatura de atributos (Colunas):.....	11
4.4.Nomenclatura de Chaves Primárias:.....	11
4.5.Nomenclatura de Chaves Estrangeiras:.....	11
5.Padrões para criação de testes de unidade.....	12
5.1.Padrões para testes Java.....	12
5.1.2.Padrões para testes React.....	12
6.Padrões para versionamento.....	13
6.1.Git Flow:.....	13
6.2.Commit:.....	14
Exemplos:.....	14
6.3.Pull Request (PR):.....	15
6.4.Issues:.....	16
TEMPLATE PADRÃO.....	17
Exemplos:.....	17
Critérios de Priorização:.....	19
PRIORIDADE ALTA:.....	19
PRIORIDADE MÉDIA.....	19
PRIORIDADE BAIXA.....	19
CRIAÇÃO DE ISSUES.....	19
DESENVOLVIMENTO.....	19
REVISÃO E FECHAMENTO.....	19
Boas práticas:.....	20
O que evitar:.....	20

1.Introdução

Este documento especifica os padrões e diretrizes a serem seguidos durante o desenvolvimento do produto para garantir a sua qualidade. As especificações se aplicam ao template das atas de reunião a serem feitas, ao código Java a ser produzido, à nomenclatura do banco de dados, à escrita dos testes e ao fluxo de uso do GitHub.

2.Ata de Reunião

Template da Ata de Reunião utilizada no Discord:

```
None
**Data X:**
**Tema:** Descrição do tema da reunião.
**Participantes:**
Participante 1, Participante 2, Participante N...
**Modalidade:** Tipo da modalidade (Presencial/Online).
```

3.Padrões de codificação em Java

3.1.Comentários

- Para comentários simples de linha única deve ser utilizado o operador de comentário simples, //, devidamente espaçado do código para melhorar a legibilidade.

```
Java
// Isso é um comentário de uma única linha.
int idade = 28; // Também pode ficar no final de uma linha de código.
```

- Para a produção de comentários de múltiplas linhas, o operador de comentário deve ser utilizado `/*...*/`.

Java

```
/*  
Esse é um comentário de múltiplas linhas.  
Você pode escrever bastante coisa aqui que não seja a documentação de classes,  
interfaces e métodos.  
Esse tipo de comentário tbm pode ser usado para comentar blocos de código  
temporariamente, ou mesmo definitivamente até que se tenha certeza de que que  
ele pode ser excluído.  
*/  
int soma = 5 + 10;
```

- Para documentar classes, interfaces e métodos deve ser usado `/**...*/`. Esse tipo de comentário é usado pela ferramenta Javadoc que usa tags especiais (como `@param`, `@return`) para extrair informações específicas e formar uma documentação do código de forma estruturada em HTML. Para isso, por exemplo, em um arquivo de nome “jogo.java”, basta colocar no terminal:

Shell

```
javadoc jogo.java -d docs
```

Isso cria uma pasta chamada “docs” com os arquivos HTML da documentação. Além disso, para documentar todos arquivos que fazem uso desse tipo de comentário, foi adicionado ao ambiente de desenvolvimento para os sêniores, o comando:

Shell

```
make maven-javadocs-completo
```

Esse documento é um facilitador descrito no *Makefile* para o comando:

Shell

```
docker compose exec backend mvn javadoc:javadoc
```

Java

```
/**
 * Calcula a soma de dois números inteiros.
 * @param a primeiro número
 * @param b segundo número
 * @return a soma de a e b
 */
public int somar(int a, int b) {
    return a + b;
}
```

3.2. Indentação e Espaçamento

- Utilizar quatro espaços para indentação ou uma tabulação (configurada para 4 espaços);
- Utilizar espaço entre operadores e operandos;
- Utilizar espaço após vírgulas;
- Utilizar espaço após o nome da Classe ou dos métodos;
- Não utilizar espaços antes de um ponto e vírgula;
- Não utilizar espaço no início e final de um parênteses - a não ser que seja necessário;

Java

```
public class Saudacao {

    public static void main(String[] args) {
        String nome = "Ana";
        System.out.println("Olá, " + nome);
    }
}
```

```
public class FormaNaoRecomendada{

    public static void main( String[] args ){
        String nome="Ana" ;
        System.out.println(" Olá,"+nome ) ;
    }
}
```

3.3.Convenções de nomenclatura

- Deve ser utilizado Camel Case como padrão de definição de nomes e não deve ser utilizado hífen (-) e nem subtraço (_);
- Utilizar nomes apenas em português o mais descritivos possíveis e por isso evitar abreviações;
- Utilizar apenas caracteres alfanuméricos, ou seja, letras (A-Z, a-z) e números (0-9);
- Para os métodos, utilizar verbos no imperativo seguidos de substantivos.

A seguir serão definidas algumas nomenclaturas específicas:

- Arquivos: O nome do arquivo deve ser iniciado com letra maiúscula e o restante deve ter letras minúsculas. Se houver mais de uma palavra no nome de tal arquivo deve-se seguir o padrão Camel Case.

```
Java
NomeArquivo.java
Nome_arquivo_nao_recomendado.java
```

- Classes e Interfaces: O nome das Classes e interfaces também devem seguir o padrão Camel Case

Java

```
// Interface
public interface AlimentacaoSaudavel {
    void comer();
}

// Classe que implementa a interface
public class PacienteNutricao implements AlimentacaoSaudavel {

    @Override
    public void comer() {
        System.out.println("A pessoa está comendo de forma saudável.");
    }

    public static void main(String[] args) {
        PacienteNutricao ana = new PacienteNutricao();
        ana.comer();
    }
}
```

- Métodos: O nome dos métodos deve ser em letras minúsculas, exceto no início de novas palavras internas, como sugere o Camel Case.

Java

```
public class ConversorTemperatura {

    public double celsiusParaFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    public double Fahrenheit_paraCelsius(double fahrenheit) { // não
recomendado
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

```

    }

    public static void main(String[] args) {
        ConversorTemperatura conversor = new ConversorTemperatura();

        double tempC = 25.0;
        double tempF = conversor.celsiusParaFahrenheit(tempC);

        System.out.println("25°C em Fahrenheit: " + tempF);
    }
}

```

- Constantes: O nome de uma constante deve ser com letras maiúsculas e é o único caso em que deve ser utilizado subtraço (_) para separar palavras.

```

Java
public class Configuracao {

    public static final int MAX_USUARIOS = 100;
    public static final String VERSAO_APP = "1.0.0";
}

```

3.4.Composição de Classes

Cada classe deve ser definida em um arquivo, ou seja, um arquivo deve possuir somente uma classe. Para a definição de uma classe, as seguintes configurações devem ser aplicadas:

- Definição da Classe;
- Variáveis:
 - Devem ser declaradas seguindo a ordem “public”, “protected”, “private”.
- Instâncias;
- Método Construtor;
- Métodos:

- Getters e Setters devem ser declarados primeiro;
- Antes de cada método deve ser feita sua documentação como especificado no item Comentários.

3.5.Regras de formatação

- Não adicionar espaço entre o método e sua lista de parâmetros;
- A abertura de chaves “{” deve ser feita no fim da mesma linha em que foi declarado o código e o fechamento deve ser feito em uma linha separada, alinhada ao conjunto do método a qual foi aberta;
- Métodos devem ser sempre separados por uma linha em branco.

Java

```
public class Calculadora {  
  
    public int somar(int a, int b){  
        return a + b;  
    }  
  
    public int multiplicar( int a, int b ) // forma não recomendada  
    {  
        return a * b;  
    }  
}
```

4.Padrões para Banco de Dados

Alguns critérios gerais devem ser seguidos ao nomear objetos no banco de dados:

- Utilizar caracteres alfanuméricos (letras (A-Z, a-z) e números (0-9));
- Não utilizar subtraço (_) ou hífen (-) para separação de palavras. A separação deve ser feita por meio da utilização de letras maiúsculas na primeira letra de novas palavras (*Camel/Case*).
- Só se deve utilizar subtraço (_) na nomenclatura de tabelas e colunas especiais, por exemplo:

- TB_Cliente → tabela de clientes
- PK_ClienteID → chave primária da tabela Cliente
- Não utilizar caracteres especiais ou acentuação;
- Não utilizar preposições (de, o, a, etc.);
- Acrônimos de siglas devem ser utilizados apenas com letras maiúsculas, por exemplo: CPF, CEP, etc.

4.1.Nomenclatura do banco de dados:

- O nome do banco de dados deve identificar o negócio ou a sigla da aplicação;
- Utilizar apenas letras minúsculas;
- Utilizar no máximo 15 caracteres;

Exemplo: dadosjogadores.

4.2.Nomenclatura de Tabelas:

- O nome de cada tabela deve ter um significado sugestivo ao entendimento da mesma.
- Utilizar o prefixo “TB ”;
- Utilizar letras maiúsculas na primeira letra de cada palavra que forme o nome da tabela;
- Evitar utilizar abreviações;
- O nome da coluna deve estar no singular;

Exemplo: TB_Pessoa (Tabela de pessoas), TB_DadosPessoais.

4.3.Nomenclatura de atributos (Colunas):

- O nome de cada atributo (coluna) deve ter um significado sugestivo ao entendimento do mesmo.
- Utilizar letras maiúsculas na primeira letra de cada palavra que forme o nome do atributo;
- O nome do atributo deve estar no singular;
- Individualizar o nome do atributo dentro do mesmo ambiente.

Exemplos: nomeFuncionario, dataPartida.

4.4.Nomenclatura de Chaves Primárias:

- Deve-se utilizar a mesma semântica utilizada para os atributos, como especificado no item anterior, adicionando apenas o prefixo “PK_”.

Exemplo: PK_Pessoa (Chave primária da tabela Pessoa).

4.5.Nomenclatura de Chaves Estrangeiras:

- Para as chaves estrangeiras, também deve-se utilizar a mesma semântica utilizada para os atributos, mas, além de adicionar “FK_”, é preciso adicionar também o nome da tabela de origem antes de identificar o nome da chave. Observe no exemplo a seguir que o nome da tabela e o nome da chave estrangeira devem estar separados por subtração (_).

Exemplo: FK_Pessoa_CPF (Chave estrangeira vinda da tabela Pessoa).

5.Padrões para criação de testes de unidade

5.1.Padrões para testes Java

O nome das classes de testes automatizados, assim como seus métodos, devem ter o prefixo “teste”, seguindo os padrões de nomenclatura já especificados anteriormente nestes respectivos itens da nomenclatura em Java. Além disso, os testes unitários automatizados devem possuir comentários de acordo com os casos de teste elaborados especificando: DADO, QUANDO e ENTÃO, como no exemplo abaixo:

```
Java
@Test
public void testeInserirAluno(){
    // DADO que o jogo é iniciado e a tela para digitar o nome foi
    acionada
```

```

    controleAluno = new ControleAluno();

    // QUANDO a função para inserir for chamada
    Aluno aluno1 = new Aluno(nomeAluno: "Teste");
    Aluno alunoInserido = controleAluno.inserirAluno(aluno1);

    // ENTÃO o aluno deve ser inserido com sucesso
    assertNotNull(alunoInserido);

    // E o nome deve ser exatamente o mesmo
    assertEquals(aluno1.getNome(), alunoInserido.getNome());
}

```

5.1.2. Padrões para testes React

Diferentemente, da sintaxe de Java, o React permite adicionar textos mais descritivos acerca dos testes automatizados efetuados. Desta maneira, o nome do arquivo usado para teste deve obedecer ao formato “NomeArquivo.test.jsx” ou “[NomeArquivo.test.js](#)” (se for usado apenas Javascript para desenvolvimento dos componentes) e junto ao corpo da função adicionar um texto descritivo e sucinto. Além disso, os testes unitários automatizados devem possuir comentários de acordo com os casos de teste elaborados especificando sempre que possível: DADO, QUANDO e ENTÃO, como no exemplo abaixo.

```

JavaScript
//DADO a tela inicial e a exibição do botão a ser clicado
test('renderiza App e BotaoClique com total inicial', async () => {
    render(<App />)

    // ENTÃO, Espera-se que o total inicial seja exibido
    await waitFor(() =>
        expect(screen.getByText('Total de Cliques: 5')).toBeInTheDocument()
    )
}

```

}

6. Padrões para versionamento

6.1. Git Flow:

Git Flow é um fluxo de trabalho para o Git para facilitar o processo de desenvolvimento. A imagem a seguir mostra a estrutura do fluxo do Git Flow e é melhor detalhada no documento **Fluxo GitFlow** ([Documento GitFlow](#)), produzido pela Gerente de Processos.

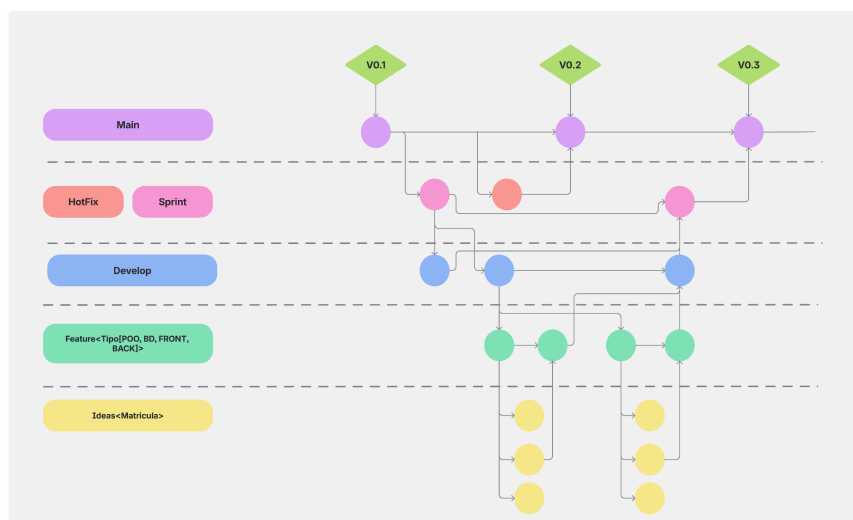


Figura 1. Git Flow a ser utilizado no projeto.

6.2. Commit:

Os commits devem ter mensagens que obedecem o padrão **tag: Síntese do que foi feito**. Sendo estas tags:

- **Feat:** Função nova;
- **Refactor:** Refatorar código existente;
- **Test:** Fazer testes automatizados;
- **Fix:** #issue<caso haja issue associada> Corrigir bug;
- **Docs:** Documentação.

Exemplos:

- **Feat:** Classe Pessoa adicionada
 - Commit pelo terminal:

Shell

```
git commit -m "Feat:Classe Pessoa adicionada"
```

- **Refactor:** Removendo métodos desnecessários da classe Ilha
 - Commit pelo terminal:

Shell

```
git commit -m "Refactor: Removendo métodos desnecessários da classe Ilha"
```

- **Test:** Adicionando teste unitário ao método solucionarDesafio da classe Desafio
 - Commit pelo terminal:

Shell

```
git commit -m "Test: Adicionando teste unitário ao método solucionarDesafio da classe Desafio"
```

- **Fix:** Removendo laço adicional no método solucionarDesafio da classe Desafio
 - Commit pelo terminal:

Shell

```
git commit -m "Fix: Removendo laço adicional no método solucionarDesafio da classe Desafio"
```

- **Fix:#25** Corrigindo instanciação de objetos na classe Ilha

- Commit pelo terminal:

Shell

```
git commit -m "Fix: #25 Corrigindo instanciação de objetos na classe Ilha "
```

- **Docs:** Adicionando documentação da classe SolucionarDesafio
 - Commit pelo terminal:

Shell

```
git commit -m "Docs: Adicionando documentação da classe SolucionarDesafio"
```

6.3.Pull Request (PR):

Sempre que uma nova tarefa for finalizada, deve ser aberta uma nova *Pull Request* descrevendo as funcionalidades integradas pela ação. Para os desenvolvedores juniores, as tarefas serão vistoriadas inicialmente pelos *Scrum Masters/Desenvolvedores Seniores* para então serem avaliadas pelos *Analistas de Qualidade*. Já para os *Desenvolvedores Seniores*, as tarefas serão testadas e revisadas pelos *Analistas de Qualidade* para então serem integradas ao repositório caso esteja tudo nos conformes.

As *PRs* seguem o formato:

- **Branch destino:** *Branch* para onde as mudanças serão direcionadas;
- **Branch origem:** *Branch* onde foram feitas as mudanças;
- **Revisores:** Pessoas que farão a revisão e aceitarão a *PR* (normalmente os *Analistas de Qualidade*);
- **Responsável:** Pessoa responsável pela atribuição da tarefa (geralmente um *Desenvolvedor Sênior*);
- **Título:** Resumo do que foi feito. Respeitando as indicações feitas nessa seção;

- **Label:** Sinaliza qual o tipo de ação está sendo efetuada na *PR*. Nem sempre haverá uma *label* que se encaixe no contexto de uma *pull request*.

6.4.Issues:

As issues devem seguir o padrão de nomenclatura:

```
None  
[TIPO]: Descrição concisa.
```

Onde TIPO pode ser:

- **Feat:** Nova funcionalidade;
- **Fix:** Correção de bug;
- **Docs:** Documentação;
- **Test:** Testes;
- **Refactor:** Refatoração.

Vale destacar que existem 4 templates no repositório para abertura de issues:

1. **Reporte de Bug:**
2. **Requisição de Feature**
3. **Requisição de Alteração**
4. **Blank issue**

Ou seja, algumas issues possuem um modelo já pronto para cenários específicos como relato de bugs, adição de funcionalidades, etc. Bastando apenas adicionar as informações como será descrito nessa seção.

TEMPLATE PADRÃO

```
None  
## Descrição  
[PRIORIDADE]  
[Descrição detalhada do objetivo]
```



```
### Contexto
[Contexto e motivação]

### Comportamento Esperado
[Resultado esperado após implementação]

##
✅ Critérios de Aceitação
- [ ] Critério 1
- [ ] Critério 2

##
Tarefas
- [ ] Tarefa 1
- [ ] Tarefa 2

##
Links Relevantes
- Link 1
- Link 2
```

Exemplos:

```
None
[FIX] - Corrigir Erro no Login

##
Descrição
Prioridade Alta
Usuários não conseguem fazer login após atualização

### Passos para Reproduzir
1. Acessar página de login
2. Preencher credenciais
3. Clicar em "Entrar"
```

Comportamento Esperado

Login deve ser realizado com sucesso

None

[FEAT] - Implementar Sistema de Notificações

##

Descrição

Prioridade Média

Implementar sistema de notificações push para usuários mobile

Contexto

Precisamos notificar usuários sobre novos eventos

Comportamento Esperado

Usuários devem receber notificações em tempo real

##

✅ Critérios de Aceitação

- [] Configuração do Firebase
- [] Implementação do serviço
- [] Testes de integração

Critérios de Priorização:

PRIORIDADE ALTA:

- Bugs em funcionalidades críticas;
- Features essenciais para release;
- Questões de segurança.

PRIORIDADE MÉDIA

- Melhorias significativas;
- Bugs em funcionalidades não críticas;
- Novas features importantes.

PRIORIDADE BAIXA

- Melhorias estéticas;
- Otimizações menores;
- Documentação não essencial.

CRIAÇÃO DE ISSUES

1. Selecionar template apropriado;
2. Preencher todas as seções obrigatórias;
3. Adicionar labels relevantes;
4. Adicionar o tipo da issue quando possível;
5. Definir prioridade - na própria descrição da issue;
6. Atribuir responsáveis quando aplicável.

DESENVOLVIMENTO

1. Criar branch seguindo padrão;
2. Referenciar número da issue nos commits;
3. Atualizar status regularmente.

REVISÃO E FECHAMENTO

1. Verificar critérios de aceitação;
2. Realizar code review quando aplicável;
3. Atualizar documentação;
4. Vincular PR (Pull request) relacionado.

Boas práticas:

- Manter descrições claras e objetivas;
- Atualizar status regularmente;
- Documentar decisões importantes;
- Usar referências cruzadas;

- Manter issues atualizadas.

O que evitar:

- Títulos vagos ou genéricos;
- Descrições incompletas;
- Falta de critérios de aceitação;
- Múltiplos problemas na mesma issue;
- Labels inconsistentes.